

Universidad de Málaga

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Proyecto de Laboratorio de Robótica GIERM 2025-2026

Programación y ensamblaje de un robot móvil
diferencial



UNIVERSIDAD DE MÁLAGA

Autor: Salvador Patricio Lázaro Herrero

Profesor: Javier Serón Barba

Fecha: 16 de enero de 2026

Índice general

1. Introducción y Objetivos	2
1.1. Contexto del Proyecto	2
1.2. Objetivos	2
2. Diseño Hardware y Esquema de Conexión	3
2.1. Componentes Utilizados	3
2.2. Esquema de Conexión	3
3. Diseño Software del Sistema	5
3.1. Arquitectura General del Sistema	5
3.2. Descripción de las tareas implementadas	5
3.2.1. Programa 1: Avance en línea recta temporizado	5
3.2.2. Programa 2: Avance rectilíneo por distancia	6
3.2.3. Programa 3: Movimiento circular	6
3.2.4. Programa 4: Avance recto con control de orientación	6
3.2.5. Programa 5: Seguimiento de trayectoria compleja	6
3.3. Algoritmo Follow the Carrot	7
3.4. Función <code>calcula_trayectoria()</code>	7
3.4.1. Cálculo de errores	7
3.4.2. Control por histéresis y estados	8
3.4.3. Ventajas del enfoque	8
3.5. Arquitectura: micro-ROS, WiFi y FreeRTOS	8
3.5.1. Comunicación WiFi con micro-ROS	9
3.5.2. Motivación del uso de FreeRTOS	9
3.5.3. Arquitectura basada en tareas FreeRTOS	9
3.5.4. Separación de control y planificación	10
3.5.5. Ventajas de la arquitectura propuesta	11
4. Resultados y Conclusiones	12
4.1. Resultados Obtenidos	12
4.2. Conclusiones y posibles mejoras	13
5. Lista y Descripción de los Ficheros Entregados	14

Capítulo 1

Introducción y Objetivos

1.1. Contexto del Proyecto

Se pretende desarrollar un robot móvil diferencial tipo piero con el objetivo de aprender competencias relacionadas en el ámbito de la robótica móvil.

1.2. Objetivos

- Diseñar y montar un sistema distribuido con sensores y actuadores.
- Implementar control local y remoto mediante comunicación Wifi.
- Realizar las 5 tareas requeridas.
- Lograr control mediante teleoperación.

Capítulo 2

Diseño Hardware y Esquema de Conexión

2.1. Componentes Utilizados

A continuación se introduce un listado de sensores, actuadores y módulos empleados para el proyecto.

Componentes	Cantidad	Descripción
Microcontrolador ESP32.	1	Modelo Dev
Driver de potencia L298N.	1	Control de los motores
Cables	X	Interconexión de dispositivos
Módulo cargador-elevador	1	Elevador 5-12V pilas 18650
Motor de corriente continua JGA25-370.	2	
Reductora	2	Reducción 35:1
Rueda con neumático.	2	Diámetro X
Sensor ultrasónico.	3	Distancia: 3m

Cuadro 2.1: Tabla de componentes.

2.2. Esquema de Conexión

Se presenta el esquemático del diseño final adjunto en la figura 2.1 que reproduce fielmente el montaje del proyecto.

Cabe destacar que en el diseño final se ha usado 1 sensor ultrasónico al contar con un espacio reducido en el chasis del robot para hacer ampliaciones. Ello debido a información posterior al diseño y ensamblaje puesto que para su interconexión es necesario hacer un divisor de tensión para reducirla a un nivel aceptable para la placa.

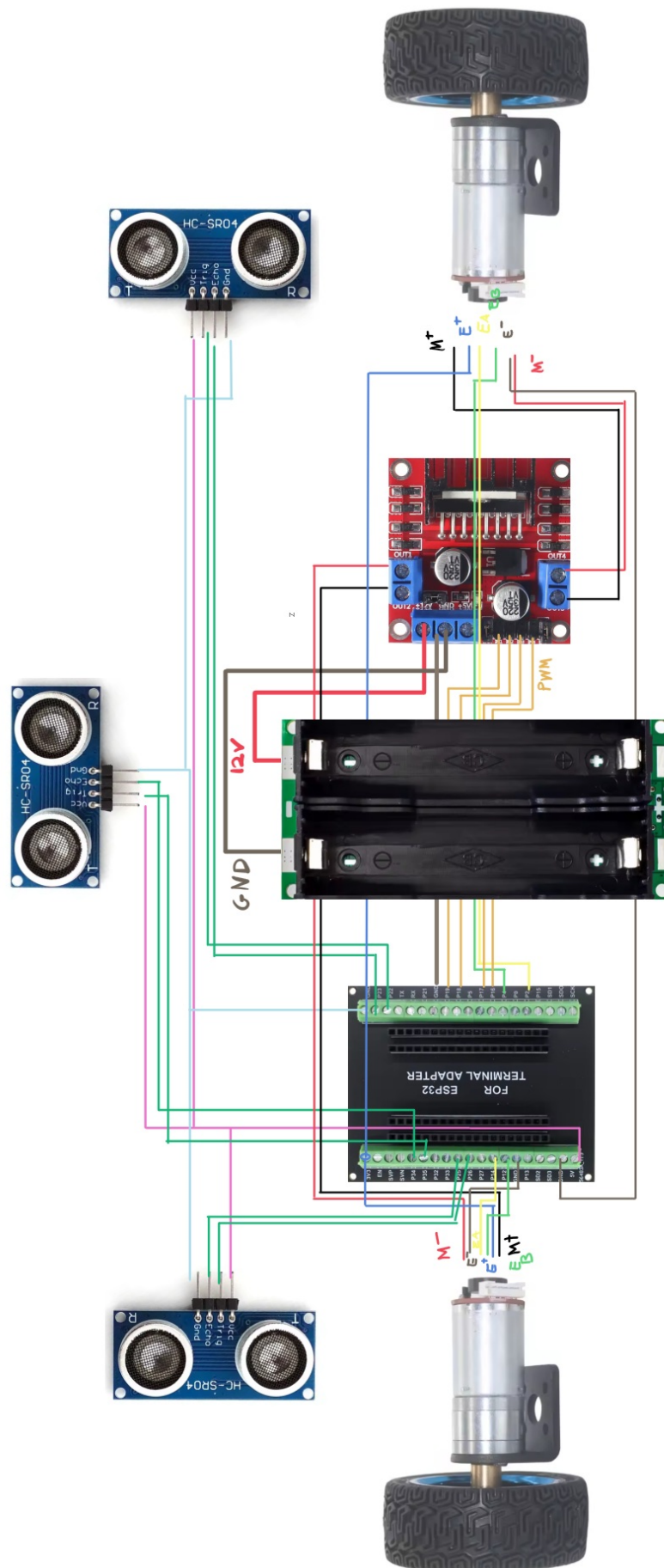


Figura 2.1: Esquema de conexión del robot móvil

Capítulo 3

Diseño Software del Sistema

3.1. Arquitectura General del Sistema

El sistema se fundamenta en una arquitectura modular, compuesta por los controladores, los programas requeridos por el profesor y los diferentes subprogramas encargados de manejar los periféricos.

En el caso de la implementación con MicroRos, en lugar de centralizar toda la lógica en un único núcleo, las funciones se han segregado en tres subsistemas especializados mediante tareas de FreeRTOS. La comunicación se hace mediante una red WiFi a través del protocolo UDP. Esta estructura permite:

- Desacoplar la lectura de sensores (inputs) de la gestión de actuadores (outputs).
- Facilitar la escalabilidad del sistema.
- Mejorar la robustez mediante el aislamiento de tareas.

3.2. Descripción de las tareas implementadas

En este proyecto se han implementado cinco tareas o programas de control que permiten validar de forma progresiva el comportamiento del robot móvil diferencial, desde movimientos simples hasta el seguimiento de trayectorias complejas.

3.2.1. Programa 1: Avance en línea recta temporizado

El **programa1** consiste en hacer avanzar el robot en línea recta durante un tiempo determinado. El contador de tiempo trabaja a una frecuencia de 2 Hz, por lo que un valor de 40 corresponde a 20 segundos.

Durante este intervalo, el robot avanza con una velocidad lineal constante de 0.1 m/s y velocidad angular nula. Una vez superado el tiempo establecido, el robot se detiene completamente.

Este programa permite verificar el correcto funcionamiento de los actuadores y la temporización básica del sistema. Sin embargo, debido a la presencia de inercia inherente al modelo, no se alcanzan los 2 metros establecidos ya que se inicia a una velocidad inferior a la necesitada.

3.2.2. Programa 2: Avance rectilíneo por distancia

El **programa2** hace que el robot avance en línea recta hasta alcanzar una distancia de 2 metros en el eje x del sistema de referencia global.

Mientras la posición actual cumpla $x < 2$, el robot se desplaza con velocidad lineal constante de 0.1 m/s y velocidad angular cero. Cuando se alcanza o supera dicha distancia, el robot se detiene.

Este programa valida el uso de la odometría para el control basado en posición puesto que se alcanzan los 2m establecidos.

3.2.3. Programa 3: Movimiento circular

El **programa3** realiza un movimiento circular de radio $R = 0,4$ m durante 10 segundos. La velocidad angular del movimiento es:

$$\omega = \frac{2\pi}{10} \text{ rad/s}$$

Durante los primeros instantes, ambas ruedas giran a la misma velocidad para iniciar el movimiento. Posteriormente, se calculan las velocidades individuales de las ruedas derecha (v_r) e izquierda (v_l) en función del radio, la velocidad angular deseada y la distancia entre ruedas.

Este programa permite comprobar el control diferencial del robot y la correcta ejecución de trayectorias curvas.

3.2.4. Programa 4: Avance recto con control de orientación

El **programa4** consiste en avanzar 2 metros en línea recta aplicando un control corrector sobre la orientación del robot.

Mientras $x < 2$, el robot avanza con velocidad lineal constante y se corrige la desviación angular mediante un controlador proporcional sobre el error de orientación:

$$\omega = K \cdot \Delta\theta$$

El signo del control depende del sentido del error angular. Este programa demuestra la capacidad del robot para mantener una orientación deseada durante el desplazamiento.

3.2.5. Programa 5: Seguimiento de trayectoria compleja

El **programa5** implementa el seguimiento de una trayectoria compuesta por varios tramos rectilíneos y giros:

- Avance recto de 1 m
- Giro de 90° a la derecha y avance de 1 m
- Giro de 180° y avance de 3 m
- Giro de 90° a la izquierda y avance de 1 m
- Giro final de 180°

La trayectoria se divide en fases, y en cada una se define un punto destino y un punto anterior que permiten calcular la orientación ideal del tramo.

El seguimiento se realiza mediante un algoritmo *Follow the Carrot*, explicado en la siguiente sección.

3.3. Algoritmo Follow the Carrot

El método **Follow the Carrot** consiste en hacer que el robot siga un punto objetivo (“zanahoria”) situado sobre la trayectoria deseada.

En cada fase del programa:

- Se define un punto destino (x_d, y_d)
- Se define un punto anterior para calcular la dirección del tramo
- Se obtiene la orientación ideal del tramo mediante:

$$\theta_{\text{ideal}} = \text{atan2}(y_d - y_{\text{ant}}, x_d - x_{\text{ant}})$$

El robot primero se orienta hacia la dirección del tramo y posteriormente avanza corrigiendo errores laterales y angulares hasta alcanzar el punto objetivo. Una vez alcanzado, se pasa a la siguiente fase de la trayectoria.

Este enfoque simplifica el seguimiento de trayectorias complejas al dividir las en segmentos rectilíneos independientes.

3.4. Función calculo_trayectoria()

La función `calculo_trayectoria()` es el núcleo del control de trayectoria y se encarga de generar las velocidades lineal y angular del robot.

3.4.1. Cálculo de errores

Se calculan los siguientes errores:

- Error de posición en x e y
- Error de distancia al objetivo
- Error lateral:

$$e_{\text{lat}} = -\sin(\theta) \cdot e_x + \cos(\theta) \cdot e_y$$

- Error angular entre la orientación actual y la orientación ideal del tramo

Todos los errores angulares se normalizan mediante la función `angleWrap`.

3.4.2. Control por histéresis y estados

El control se basa en una máquina de estados con histéresis para evitar oscilaciones:

- **Estado 0 – Giro en el sitio:** El robot gira sobre sí mismo hasta alinearse con la orientación ideal del tramo. La salida del estado depende de un margen que varía según el ángulo requerido.
- **Estado 1 – Transición:** Se introduce un pequeño retardo para suavizar el cambio entre giro y avance, descargando integradores y evitando transitorios bruscos.
- **Estado 2 – Avance con corrección:** El robot avanza hacia el objetivo aplicando una corrección angular proporcional al error angular y al error lateral:

$$\omega = K_1 \cdot \Delta\theta + K_2 \cdot e_{lat}$$

Cuando la distancia al objetivo es menor que un umbral definido, el robot se detiene y pasa a la siguiente fase de la trayectoria.

3.4.3. Ventajas del enfoque

Este método permite:

- Separar claramente giro y avance
- Reducir oscilaciones mediante histéresis
- Seguir trayectorias complejas de forma robusta
- Facilitar la ampliación a trayectorias más largas o dinámicas

3.5. Arquitectura: micro-ROS, WiFi y FreeRTOS

Para conseguir un control remoto del robot, se ha elegido separar las primeras 5 tareas de la última, que introduce el control a distancia del robot.

El sistema desarrollado se basa en una arquitectura distribuida que integra un microcontrolador ESP32 con ROS 2 mediante **micro-ROS**, utilizando comunicación WiFi y un sistema operativo en tiempo real (**FreeRTOS**) para garantizar el cumplimiento de restricciones temporales estrictas.

El objetivo principal de esta arquitectura es permitir:

- Teleoperación del robot desde un nodo remoto en ROS 2 mediante comandos **Twist**
- Publicación continua de la odometría del robot
- Ejecución determinista de los controladores PID a 20 ms
- Implementación del seguimiento de trayectoria (Programa 5) de forma remota

3.5.1. Comunicación WiFi con micro-ROS

La comunicación entre el ESP32 y el sistema ROS 2 se realiza mediante **micro-ROS sobre WiFi**. El ESP32 actúa como un nodo ROS 2 embebido que se conecta a un *micro-ROS agent* ejecutándose en un ordenador remoto.

La inicialización del transporte WiFi se realiza mediante:

```
set_microros_wifi_transports()
```

Una vez establecida la conexión, el ESP32 se comporta como un nodo ROS 2 estándar, pudiendo:

- Suscribirse al tópico `/cmd_vel` (mensajes `geometry_msgs/Twist`)
- Publicar la odometría en el tópico `/cmd_pose` (mensajes `geometry_msgs/Pose`)

3.5.2. Motivación del uso de FreeRTOS

Durante el desarrollo inicial, el control del robot se realizaba dentro del bucle principal `loop()`. Sin embargo, esta aproximación presentó problemas graves de **jitter temporal** debido a:

- Gestión de red WiFi
- Ejecución del executor de micro-ROS
- Recepción de mensajes ROS 2

Como consecuencia, el tiempo de ejecución del controlador PID, que debía ser de 20 ms, llegó a degradarse hasta valores del orden de **950 ms**, haciendo imposible un control estable de velocidad.

Para resolver este problema se optó por utilizar **FreeRTOS**, separando explícitamente las tareas críticas en tiempo real de las tareas no deterministas.

3.5.3. Arquitectura basada en tareas FreeRTOS

El sistema se estructura en varias tareas concurrentes, cada una con una responsabilidad bien definida:

Tarea de control (Task_Control)

Esta tarea es la más crítica del sistema y se ejecuta:

- Con prioridad alta
- Fijada al núcleo 0 del ESP32
- Activada de forma periódica cada 20 ms mediante un temporizador hardware

Dentro de esta tarea se realizan:

- Lectura de encoders
- Cálculo de odometría

- Integración de la posición (x, y, θ)
- Cálculo de velocidades
- Ejecución de los controladores PID de velocidad
- Generación de señales PWM para los motores

El uso de un **timer hardware** junto con un **semaforo** garantiza una ejecución determinista, eliminando el jitter observado en el bucle principal.

Tarea micro-ROS (Task_MicroROS)

Esta tarea se encarga exclusivamente de:

- Ejecutar el `rclcpp_executor`
- Procesar callbacks de suscriptores
- Gestionar la comunicación ROS 2

Se ejecuta con prioridad baja en el núcleo 1, ya que no tiene requisitos temporales estrictos y su latencia no afecta directamente a la estabilidad del control.

Tarea de sensor ultrasónico (Task_Sonar)

Esta tarea se ejecuta de forma periódica a 2 Hz y realiza mediciones de distancia mediante sensor ultrasónico, también controla la activación de un flag de parada de emergencia y por último hace una publicación periódica de la odometría.

Su prioridad es intermedia, ya que no es crítica para el control, pero sí para la seguridad del sistema.

3.5.4. Separación de control y planificación

Un aspecto clave de la arquitectura es la separación entre:

- **Control de bajo nivel** (velocidad, PID, motores), ejecutado localmente en el ESP32
- **Planificación y decisión**, ejecutada remotamente en ROS 2

El nodo remoto en ROS 2 envía comandos de velocidad mediante mensajes `Twist`. El ESP32 se limita a ejecutar de forma robusta y determinista los comandos recibidos, garantizando estabilidad y seguridad.

3.5.5. Ventajas de la arquitectura propuesta

La arquitectura adoptada presenta las siguientes ventajas:

- Eliminación del jitter en el control PID
- Cumplimiento estricto del periodo de muestreo de 20 ms
- Integración completa con el ecosistema ROS 2
- Capacidad de teleoperación y control remoto
- Escalabilidad hacia navegación autónoma
- Separación clara entre tiempo real y comunicaciones

Esta solución permite combinar las ventajas de ROS 2 con las restricciones de un sistema embebido en tiempo real, logrando un sistema robusto y extensible.

Capítulo 4

Resultados y Conclusiones

4.1. Resultados Obtenidos

El proyecto desarrollado ha cumplido satisfactoriamente con los objetivos y especificaciones planteadas inicialmente, demostrando el correcto funcionamiento de un robot móvil diferencial completamente operativo tanto a nivel de hardware como de software. A lo largo del desarrollo se ha logrado integrar de manera exitosa el control de motores, la lectura de encoders, la estimación de la posición mediante odometría y la ejecución de diferentes programas de movimiento y seguimiento de trayectorias.

Desde el punto de vista del control, el uso de reguladores PID para la velocidad angular de cada rueda ha permitido un comportamiento estable y preciso del robot. La incorporación de técnicas como la compensación de zona muerta, la limitación de saturación y el anti-windup ha contribuido a mejorar la respuesta del sistema, evitando oscilaciones y errores acumulados en el control. Los resultados experimentales muestran que el robot es capaz de seguir consignas de velocidad lineal y angular de forma suave, manteniendo una buena estabilidad incluso durante cambios de trayectoria y maniobras de giro en el sitio.

En cuanto a la localización, la odometría implementada a partir de los encoders proporciona una estimación coherente de la posición (x,y) y de la orientación. Aunque este método presenta las limitaciones inherentes a la acumulación de error, los resultados obtenidos son adecuados para trayectorias cortas y medias, cumpliendo con las exigencias del proyecto. Además, el seguimiento de trayectorias por fases, con control de orientación y corrección lateral, ha demostrado ser eficaz para ejecutar recorridos compuestos por tramos rectos y giros definidos.

La integración del sensor ultrasónico añade una capa básica de seguridad, permitiendo la detección de obstáculos y la detención del robot cuando se encuentra un objeto a una distancia inferior al umbral definido. Este comportamiento refuerza la robustez del sistema y demuestra la viabilidad de combinar control de movimiento con percepción del entorno.

En conjunto, los resultados obtenidos confirman que el robot diferencial desarrollado es funcional, estable y fiable, por lo que puede afirmarse que el proyecto ha sido un éxito y que se han cumplido las especificaciones solicitadas.

4.2. Conclusiones y posibles mejoras

Como conclusión, se puede afirmar que el proyecto ha sido un éxito, ya que se ha conseguido implementar un sistema completo, funcional y bien estructurado, combinando electrónica, comunicaciones y programación. El uso de FreeRTOS ha permitido gestionar múltiples tareas de forma eficiente, y UDP ha demostrado ser un protocolo adecuado para la comunicación entre nodos distribuidos.

No obstante, existen diversas líneas de mejora y ampliación que podrían abordarse en trabajos futuros:

- incorporación de nuevos sensores, como encoders de mayor resolución, sensores inerciales (IMU) o un sensor LiDAR que permita una percepción más rica del entorno.
- mejora del sistema de alimentación mediante una batería de mayor capacidad o con sistemas de gestión energética avanzados permitiría aumentar la autonomía del robot.
- implementar algoritmos más avanzados de localización y navegación, como fusión sensorial, mapeado del entorno (SLAM) y planificación de trayectorias autónoma.
- La integración completa con ROS o micro-ROS abriría la puerta a un desarrollo más modular y escalable, facilitando la comunicación con otros sistemas y el uso de herramientas de visualización y depuración.

En conclusión, el robot diferencial desarrollado constituye una plataforma sólida y versátil, adecuada tanto para fines académicos como experimentales, y representa una base excelente sobre la que seguir construyendo sistemas de navegación autónoma más complejos.

Capítulo 5

Lista y Descripción de los Ficheros Entregados

- **main.ino**: Código del ESP32.
- **carpeta**: Paquete de Ros2 (carpeta src)
- **memoria.pdf**: Documento final de la memoria.
- **videos.mp4**: Videos de demostración.